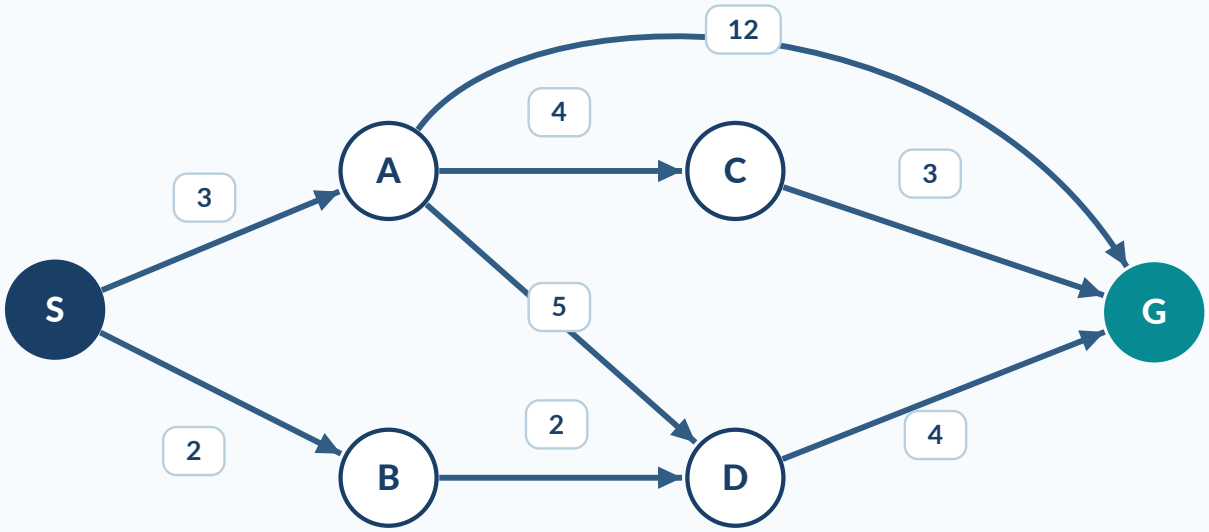
DFS & UCS - Numerical Practice

Practice 01 - Shortcut vs Cheapest Route

Q1. Apply uninformed search to the directed, weighted graph.

Start: S Goal: G All step costs are positive Expand children alphabetically Follow arrow direction.

WEIGHTED SEARCH GRAPH



ONE QUESTION | FOUR PARTS

(a) DEPTH-FIRST SEARCH (DFS) — 3 marks

Write node removal order (include G), first path to G, and its total cost.

(b) UNIFORM-COST SEARCH (UCS) — 4 marks

Show node removal order with g(n), including G. Give the cheapest path and its total cost.

SEARCH RULES

DFS/IDDFS: children A-Z; avoid cycles along the current search path.
UCS: lowest g(n), then node name; retain lower g; test goal on removal.

(c) ITERATIVE DEEPENING (IDDFS) — First successful depth limit + returned path. — 1 mark

(d) COMPARISON TABLE — General tree-search properties (not just this finite graph). — 2 marks

Algorithm	Complete?	Cost-optimal?	Time complexity	Space complexity
DFS				
UCS				
Iterative Deepening				

Use general guarantees; "optimal" means minimum path cost. Assume finite branching and costs >= eps > 0 for UCS.

Notation: b = branching factor, d = shallowest goal depth, m = maximum depth, C^* = optimal cost, eps = minimum edge cost.

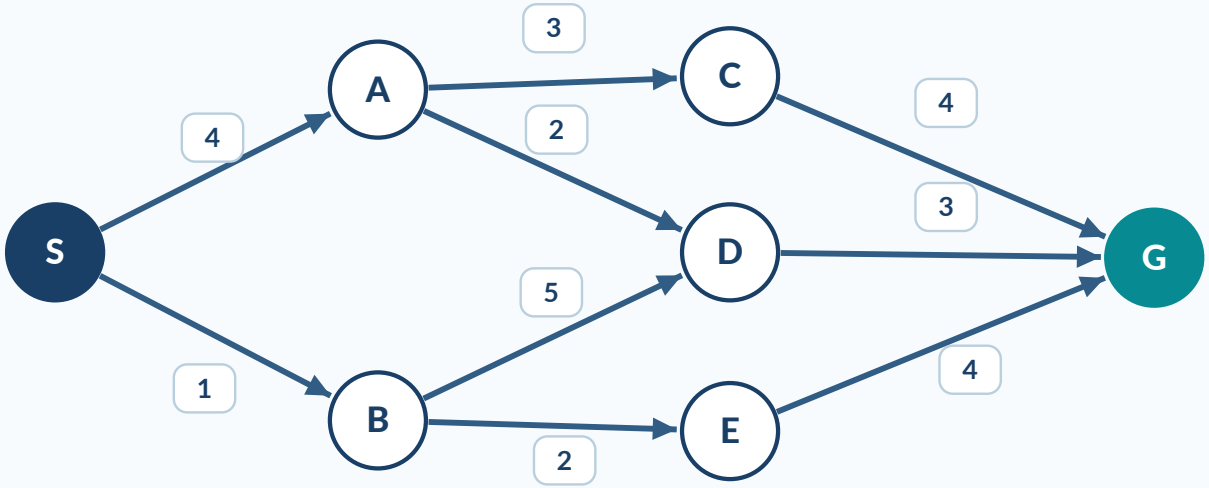
DFS & UCS - Numerical Practice

Practice 02 - Competing Weighted Routes

Q1. Apply uninformed search to the directed, weighted graph.

Start: S Goal: G All step costs are positive Expand children alphabetically Follow arrow direction.

WEIGHTED SEARCH GRAPH



ONE QUESTION | FOUR PARTS

(a) DEPTH-FIRST SEARCH (DFS) — 3 marks

Write node removal order (include G), first path to G, and its total cost.

(b) UNIFORM-COST SEARCH (UCS) — 4 marks

Show node removal order with g(n), including G. Give the cheapest path and its total cost.

SEARCH RULES

DFS/IDDFS: children A-Z; avoid cycles along the current search path.
UCS: lowest g(n), then node name; retain lower g; test goal on removal.

(c) ITERATIVE DEEPENING (IDDFS) — First successful depth limit + returned path. — 1 mark

(d) COMPARISON TABLE — General tree-search properties (not just this finite graph). — 2 marks

Algorithm	Complete?	Cost-optimal?	Time complexity	Space complexity
DFS				
UCS				
Iterative Deepening				

Use general guarantees; "optimal" means minimum path cost. Assume finite branching and costs >= eps > 0 for UCS.

Notation: b = branching factor, d = shallowest goal depth, m = maximum depth, C^* = optimal cost, eps = minimum edge cost.

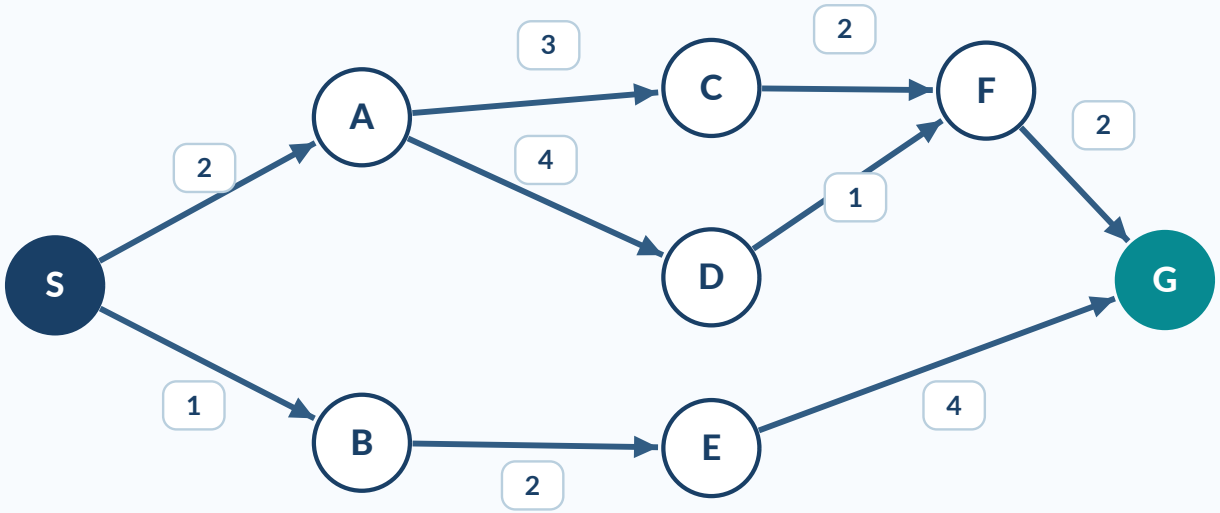
DFS & UCS - Numerical Practice

Practice 03 - Depth Versus Path Cost

Q1. Apply uninformed search to the directed, weighted graph.

Start: S Goal: G All step costs are positive Expand children alphabetically Follow arrow direction.

WEIGHTED SEARCH GRAPH



ONE QUESTION | FOUR PARTS

(a) DEPTH-FIRST SEARCH (DFS) — 3 marks

Write node removal order (include G), first path to G, and its total cost.

(b) UNIFORM-COST SEARCH (UCS) — 4 marks

Show node removal order with g(n), including G. Give the cheapest path and its total cost.

SEARCH RULES

DFS/IDDFS: children A-Z; avoid cycles along the current search path.
UCS: lowest g(n), then node name; retain lower g; test goal on removal.

(c) ITERATIVE DEEPENING (IDDFS) — First successful depth limit + returned path. — 1 mark

(d) COMPARISON TABLE — General tree-search properties (not just this finite graph). — 2 marks

Algorithm	Complete?	Cost-optimal?	Time complexity	Space complexity
DFS				
UCS				
Iterative Deepening				

Use general guarantees; "optimal" means minimum path cost. Assume finite branching and costs >= eps > 0 for UCS.

Notation: b = branching factor, d = shallowest goal depth, m = maximum depth, C^* = optimal cost, eps = minimum edge cost.

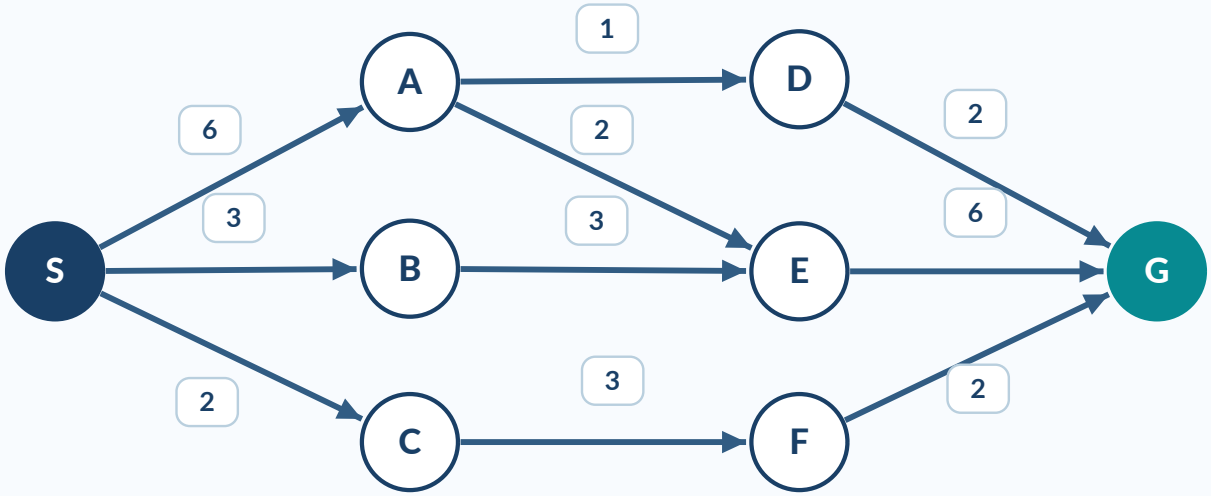
DFS & UCS - Numerical Practice

Practice 04 - Three-Way Branching

Q1. Apply uninformed search to the directed, weighted graph.

Start: S Goal: G All step costs are positive Expand children alphabetically Follow arrow direction.

WEIGHTED SEARCH GRAPH



ONE QUESTION | FOUR PARTS

(a) DEPTH-FIRST SEARCH (DFS) — 3 marks

Write node removal order (include G), first path to G, and its total cost.

(b) UNIFORM-COST SEARCH (UCS) — 4 marks

Show node removal order with g(n), including G. Give the cheapest path and its total cost.

SEARCH RULES

DFS/IDDFS: children A-Z; avoid cycles along the current search path.
UCS: lowest g(n), then node name; retain lower g; test goal on removal.

(c) ITERATIVE DEEPENING (IDDFS) — First successful depth limit + returned path. — 1 mark

(d) COMPARISON TABLE — General tree-search properties (not just this finite graph). — 2 marks

Algorithm	Complete?	Cost-optimal?	Time complexity	Space complexity
DFS				
UCS				
Iterative Deepening				

Use general guarantees; "optimal" means minimum path cost. Assume finite branching and costs >= eps > 0 for UCS.

Notation: b = branching factor, d = shallowest goal depth, m = maximum depth, C^* = optimal cost, eps = minimum edge cost.

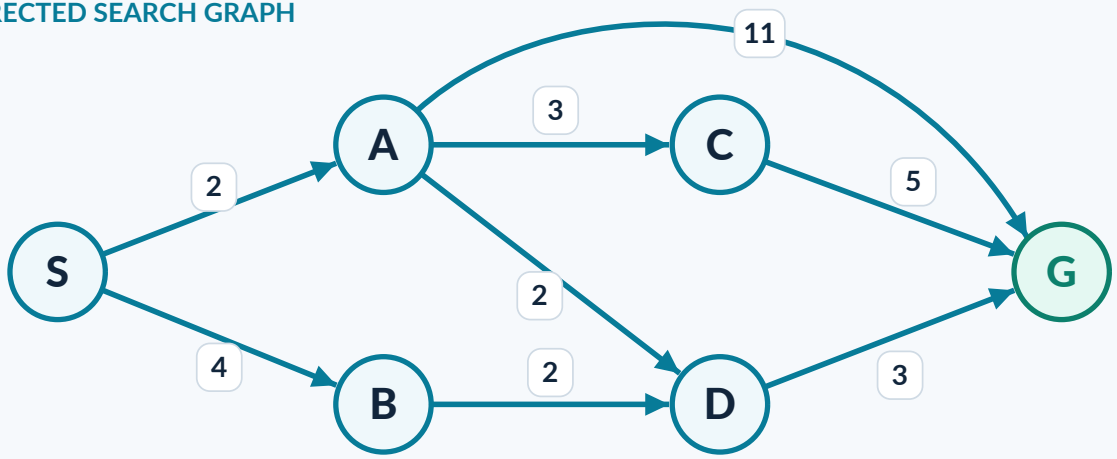
Uninformed Search - Numerical Quiz

Q1. Solve the following directed, positively weighted search graph.

Start = S; Goal = G. Explore successors alphabetically for DFS and IDDFS.

For UCS, choose the least g-cost, break ties alphabetically, retain cheaper paths, and test G when popped.

DIRECTED SEARCH GRAPH



Arrow direction is mandatory. Upper curved edge: A -> G (11).

ONE QUESTION - FOUR PARTS

(a) DFS — **3 marks**

Give the visit order through G, first DFS path, and its total step cost.

(b) UCS — **4 marks**

Trace frontier removals with g(n), then give the minimum-cost S-to-G path and cost.

For this trace, ignore a costlier path to a state already reached more cheaply.

(c) IDDFS First successful depth limit and returned path. **1 mark**

(d) Comparison table **2 marks**

Algorithm	Complete?	Optimal?	Time complexity	Space complexity
DFS				
UCS				
Iterative Deepening				

(d) Use standard TREE-SEARCH worst-case bounds (not the finite-graph bound); state any optimality conditions.

Notation: b = max branching factor; d = shallowest goal depth; m = maximum search depth; C* = optimal path cost; e = smallest step cost > 0.

Greedy Best-First & A*

13 PROBLEMS

BEGINNER TO CHALLENGE

Student: _____ Date: _____

GREEDY BEST-FIRST

priority(n) = h(n)

A* SEARCH

f(n) = g(n) + h(n)

g = actual cost so far | h = estimated remaining cost | goal h = 0

Rules: Directed edges; expand the smallest-priority node. Break equal priorities alphabetically. Stop when the goal is removed from OPEN. Keep each node's best g and parent; update better routes, including reopening CLOSED nodes.

1 Knowledge check: true or false

7 ITEMS

#	Statement	T / F
a	Greedy Best-First orders OPEN using h alone.	
b	A* orders OPEN using g + h.	
c	Greedy is guaranteed to find a least-cost route.	
d	An improved path to a node should update its g and parent.	
e	Consistent heuristics are also admissible (with h(goal) = 0).	
f	A* should stop as soon as the goal is generated.	
g	With a consistent h, A* graph search need not reopen CLOSED nodes.	

2 Compute and rank priorities

CALCULATION

Complete f = g + h. Then list the complete pop-order for each strategy.

Node	g	h	f = g + h
A	4	3	
B	2	5	
C	7	1	
D	3	3	

Greedy ranking: _____ > _____ > _____ > _____

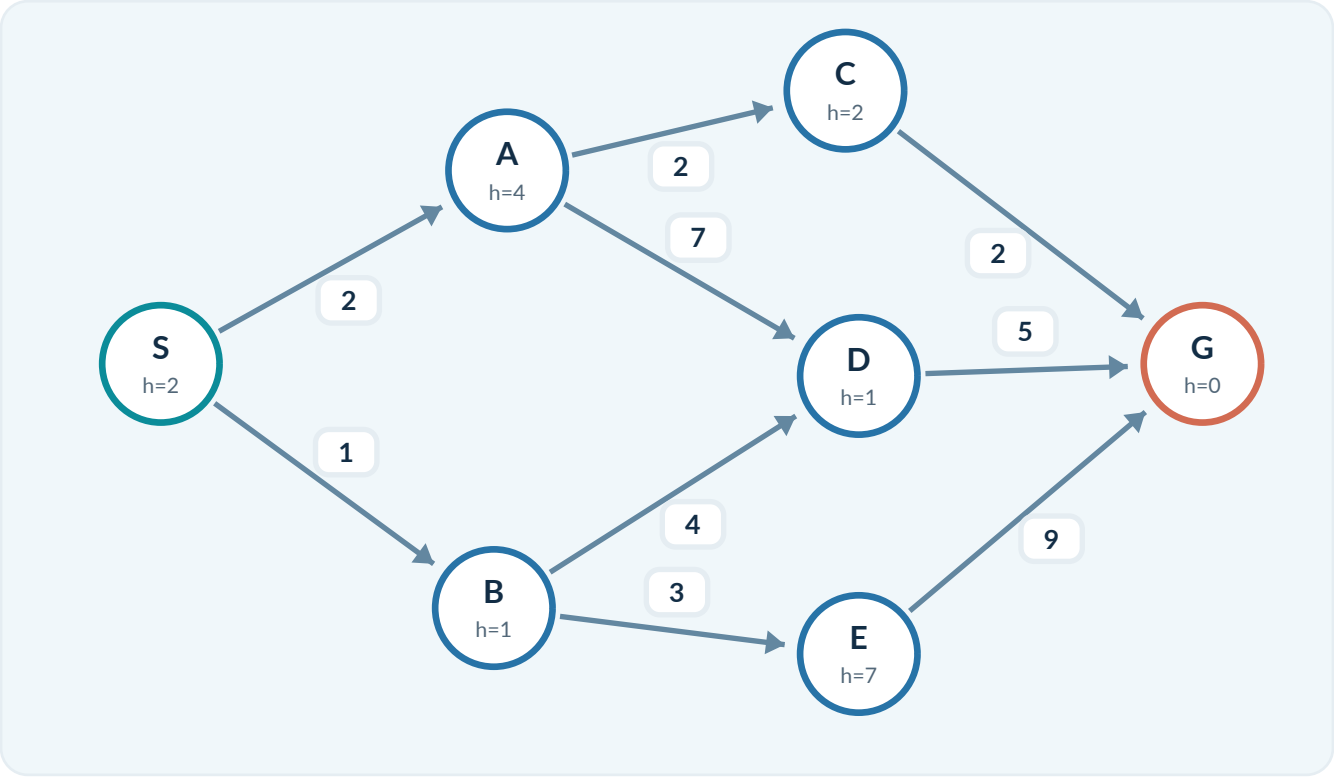
A* ranking: _____ > _____ > _____ > _____

Trace a directed weighted graph

3 Greedy Best-First on Graph A

TRACE

From S to G, trace OPEN using priority h(n) only. Edge numbers are travel costs; h values are inside each node.



Record the popped node, its accumulated g, and OPEN after adding successors.

Write OPEN in pop-order (example: B, A, D); show only each node's best record.

Step	Popped	g	h	OPEN after expansion
1				
2				
3				
4				
5				

Path S -> G: _____

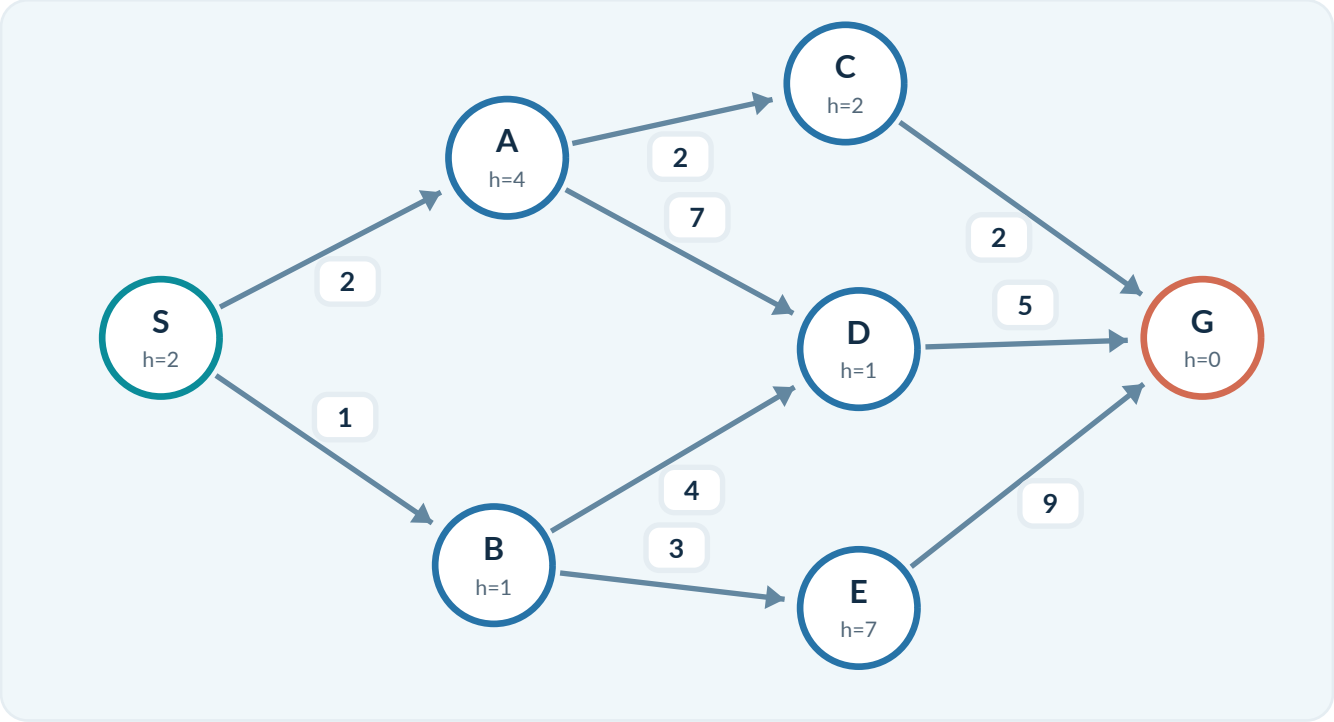
Total path cost: _____ Is this necessarily optimal? Explain. _____

Use path costs and heuristic values

4 A* on Graph A

TRACE

Use the same graph, but pop the node with smallest f = g + h. Keep improved parent pointers and best g values.



Record OPEN in increasing (f, alphabetical-name) order after each expansion.

Step	Popped	g	h	f	OPEN after expansion
1					
2					
3					
4					
5					
6					
7					

Optimal path: _____

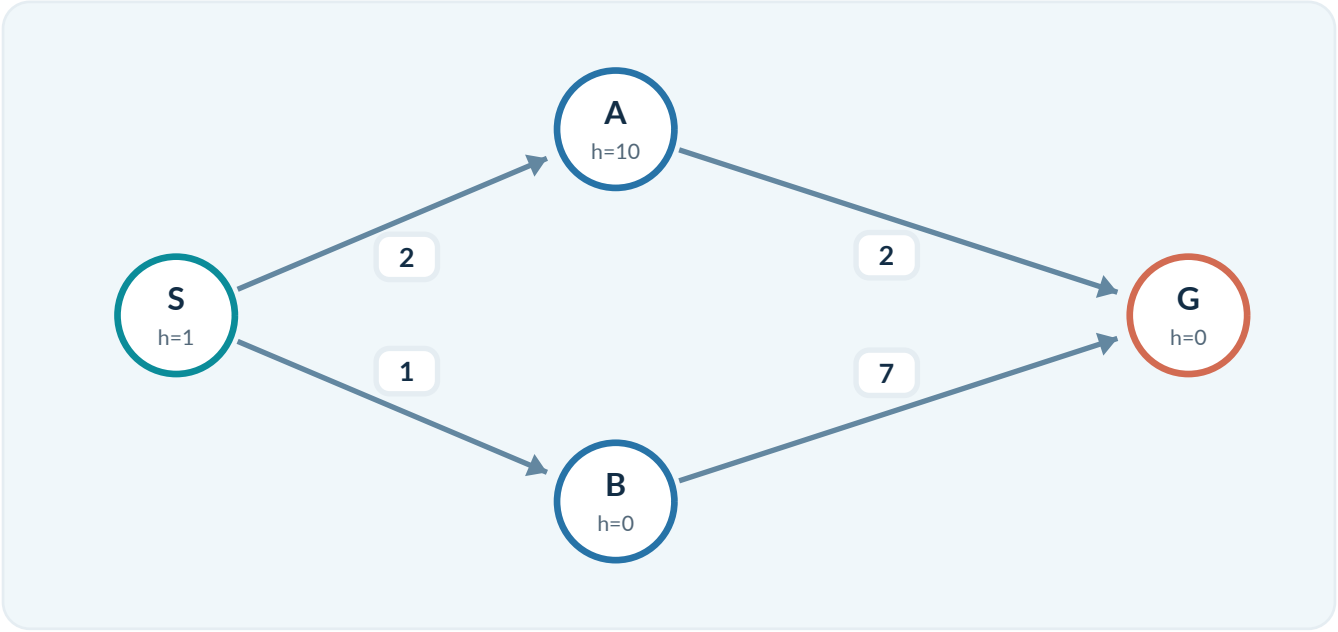
Total cost: _____ Which algorithm finds a cheaper route here? _____

When an estimate is too high

5 An overestimating heuristic

COMPARISON

Run both algorithms on Graph B. This time h(A) is intentionally unreliable. Record the pop-order, chosen path, and total cost.



Greedy Best-First

Step	Pop	g
1		
2		
3		
4		

A* Search

Step	Pop	f
1		
2		
3		
4		

Greedy path / cost:

A* path / cost:

Find the truly cheapest S -> G route and its cost:

Which h value violates admissibility? Give its true remaining cost:

Explain in one sentence why A* is not guaranteed optimal with this h:

Priority queues and improved routes

6 Sort the OPEN list

SHORT ANSWER

Entries below are already in OPEN. Sort by each algorithm's priority.

Node	g	h	f (fill in)
A	6	1	
B	2	4	
C	4	2	
D	1	4	

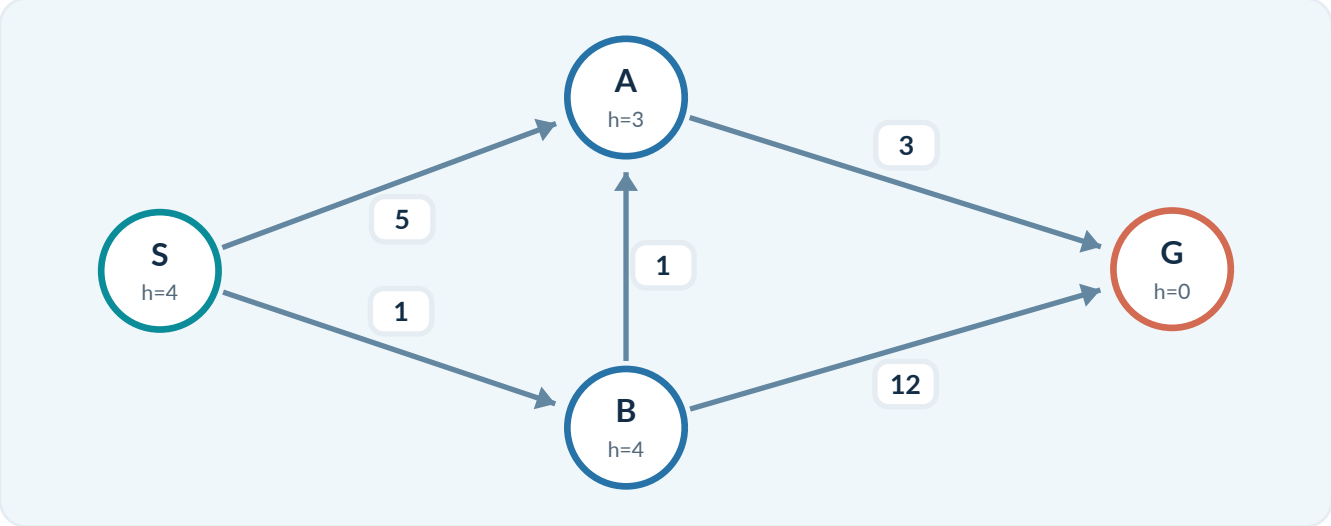
Greedy order: _____ > _____ > _____ > _____

A* order: _____ > _____ > _____ > _____

7 Update a cheaper discovered route

TRACE

Graph C: run A* from S to G. When B finds a better path to A, replace the old g(A) and parent(A).



Trace OPEN (node, f) after expansion. Do not retain stale queue records.

Step	Popped	g	OPEN after expansion
1			
2			
3			
4			

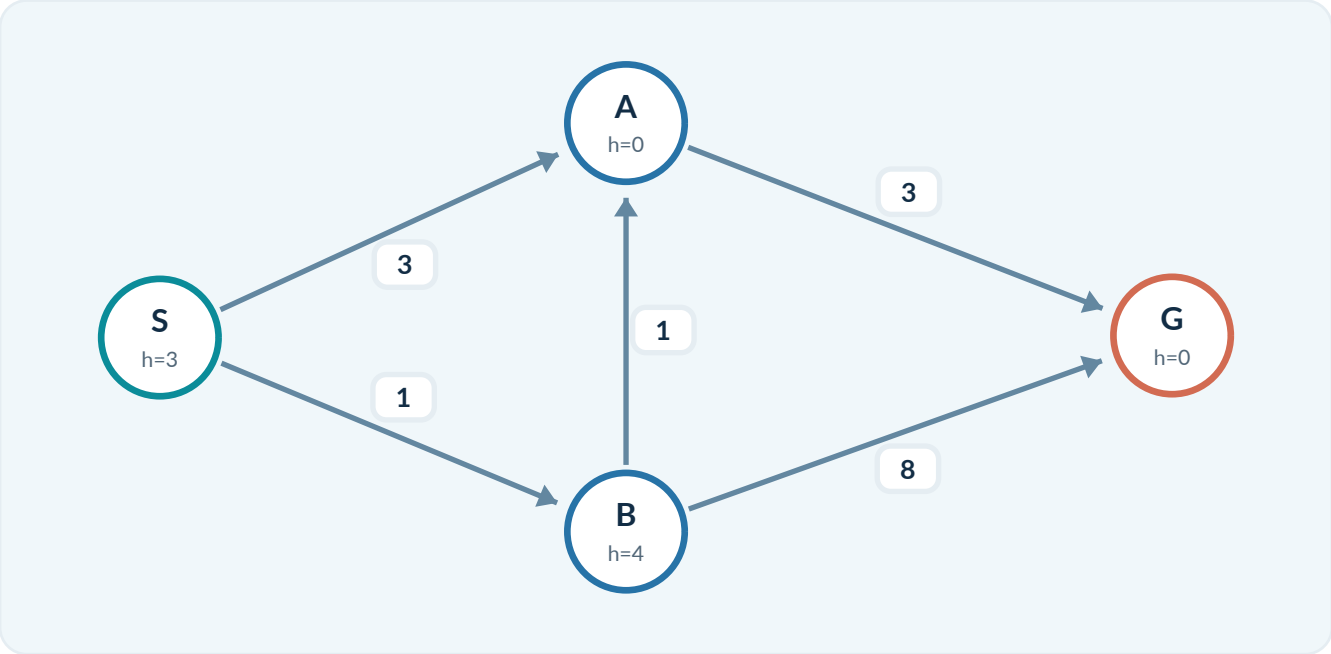
Final route: _____ Cost: _____

Inconsistent heuristics and reopening

8 A* must reopen a CLOSED node

CHALLENGE

Graph D has an admissible but inconsistent heuristic. Use the stated reopen rule when a strictly better g is discovered.



Complete the trace; the same node may appear in the Popped column twice.

Step	Popped	g	h	f	OPEN after expansion
1					
2					
3					
4					
5					
6					

Which node is reopened? _____ Old g: _____ New g: _____

Result with reopening: path _____ cost _____

If CLOSED nodes cannot reopen, which inferior result would be returned?

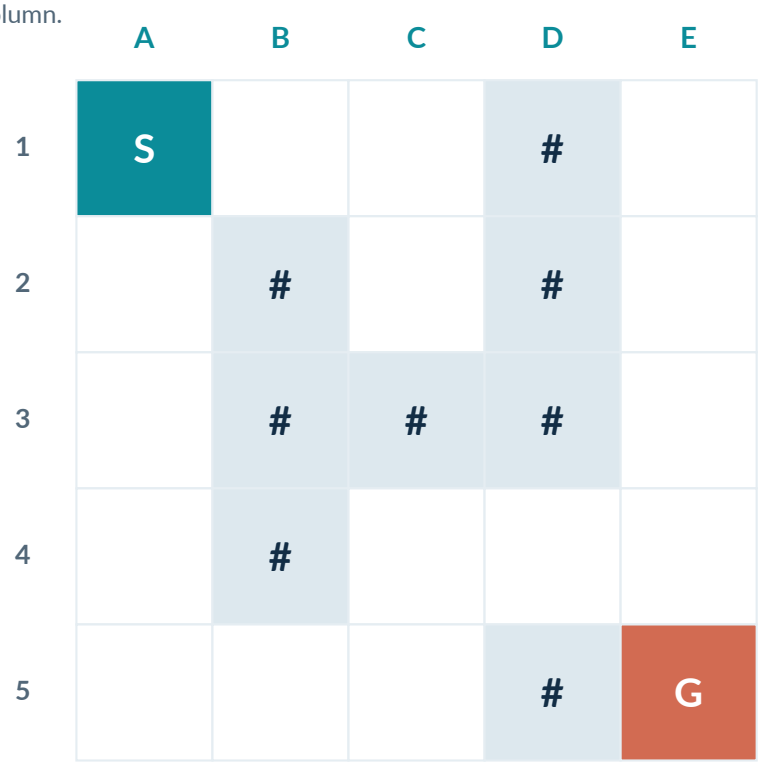
Identify an edge that violates consistency: _____

Manhattan distance and obstacles

9 Navigate a blocked grid

APPLIED

Move up/down/left/right; cost 1 per move. # is blocked. S = A1 and G = E5. Use Manhattan h and grid ties by row, then column.



HEURISTIC

h = |col - E|

+ |row - 5|

Remember: Manhattan ignores walls.

Fill estimated distances:

h(A1) = ____

h(C2) = ____

h(C5) = ____

h(E4) = ____

Mark the shortest path on the grid, then answer:

Shortest path (cell sequence):

Path cost: _____ Manhattan h(S): _____ Are they equal? _____

First six popped cells (include S):

Greedy: _____ > _____ > _____ > _____ > _____ > _____

A*: _____ > _____ > _____ > _____ > _____ > _____

Why can Manhattan underestimate the actual remaining distance?

Admissibility vs. consistency

10 Audit two heuristic functions

ANALYSIS

All edges below are directed. First calculate the true shortest distance d*(n,G); then evaluate the two heuristic columns.

DIRECTED EDGES

From	To	Cost
S	P	1
S	R	2
P	Q	2
P	G	5
R	Q	2
R	G	8
Q	G	3

ESTIMATES

Node	h1	h2
S	6	5
P	4	5
R	6	4
Q	3	1
G	0	0

Check all outgoing edges

TRUE REMAINING COSTS

d*(S)	d*(P)	d*(R)	d*(Q)	d*(G)

Classify each estimate (write Yes / No):

	h1	h2
Admissible?		
Consistent?		

Which node(s) have h1 > d*?

Give a directed edge violating consistency for h1:

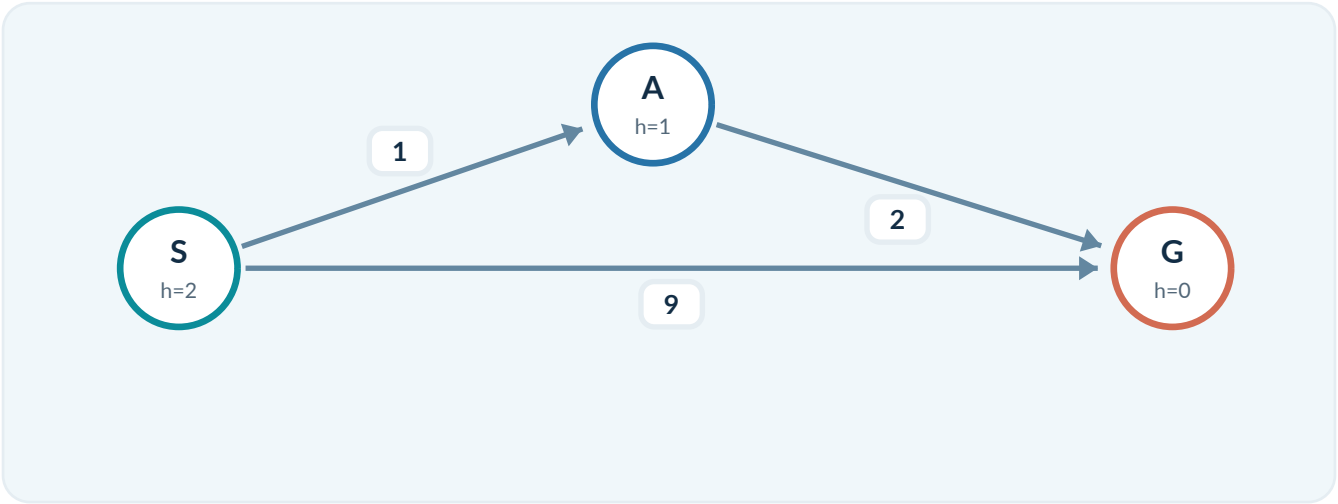
Give a directed edge violating consistency for h2:

Stopping rules and heuristic design

11 Do not stop when G is generated

DEBUGGING

An implementation stops at the first discovery of G. Show why it is wrong for this directed graph (use A*).



Early-stop path and cost: _____

Correct A* pop order: _____

Correct best path and cost: _____

12 Judge candidate grid heuristics

CONCEPT

Assume 4-direction unit-cost movement to goal G, with possible blocked cells.

Heuristic	Admissible?	Consistent?
h(n) = 0		
h(n) = Manhattan(n,G)		
h(n) = 2 x Manhattan(n,G)		
h(n) = max(0, Manhattan(n,G)-1)		
h(n) = Euclidean straight-line distance		

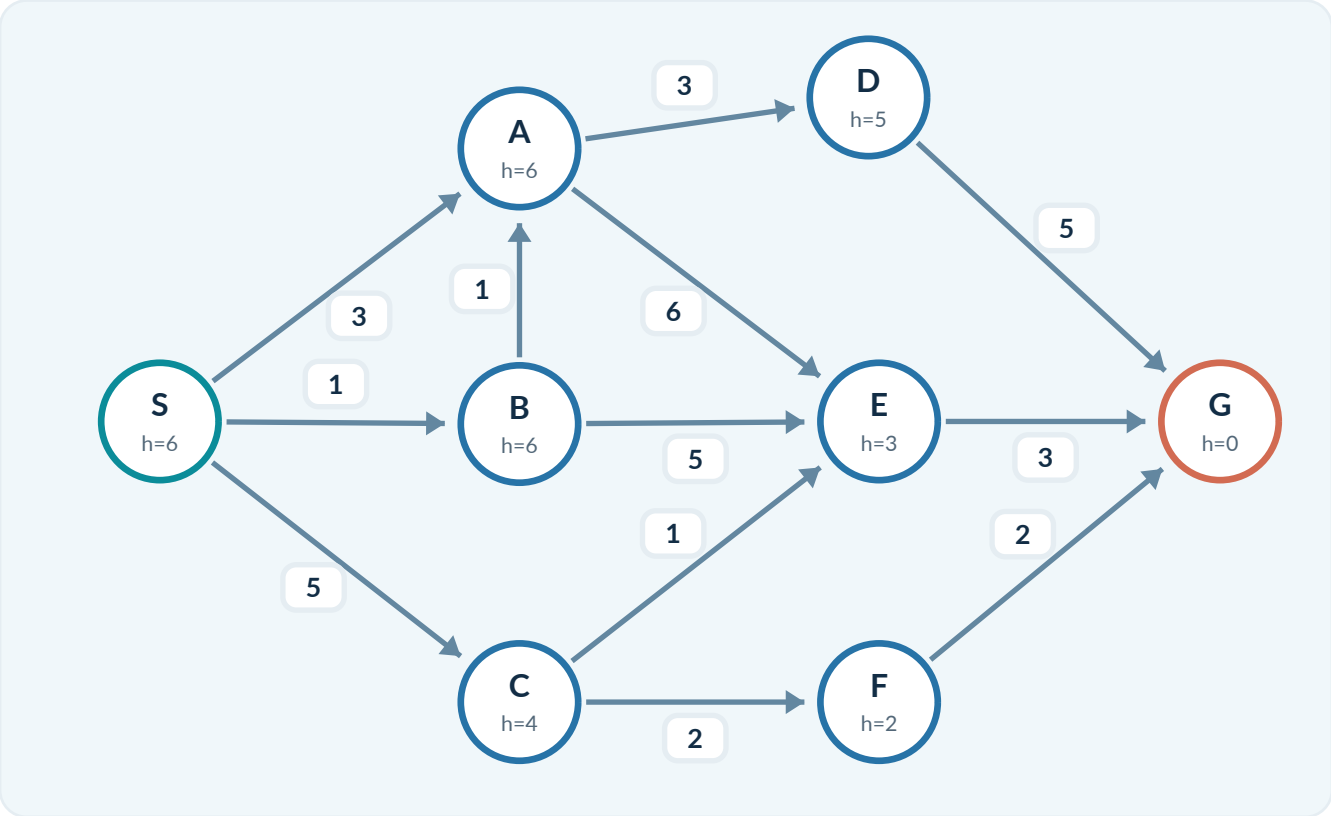
One sentence: what is the key difference between admissible and consistent?

Complete an A* run and explain ties

13 Capstone: Graph E

FINAL CHALLENGE

Trace A* from S to G. A better route to A is found later, and more than one optimal route exists.



List popped nodes and each state's g and f; use alphabetical tie-breaking.

Step	Popped	g	f	Best parent
1				
2				
3				
4				
5				
6				
7				

Expansion order: _____

Returned path and cost: _____

A different equally optimal route: _____

Greedy Best-First & A*

13 PROBLEMS

BEGINNER TO CHALLENGE

Student: _____ Date: _____

GREEDY BEST-FIRST

priority(n) = h(n)

A* SEARCH

f(n) = g(n) + h(n)

g = actual cost so far | h = estimated remaining cost | goal h = 0

Rules: Directed edges; expand the smallest-priority node. Break equal priorities alphabetically. Stop when the goal is removed from OPEN. Keep each node's best g and parent; update better routes, including reopening CLOSED nodes.

1 Knowledge check: true or false

7 ITEMS

#	Statement	T / F
a	Greedy Best-First orders OPEN using h alone.	
b	A* orders OPEN using g + h.	
c	Greedy is guaranteed to find a least-cost route.	
d	An improved path to a node should update its g and parent.	
e	Consistent heuristics are also admissible (with h(goal) = 0).	
f	A* should stop as soon as the goal is generated.	
g	With a consistent h, A* graph search need not reopen CLOSED nodes.	

2 Compute and rank priorities

CALCULATION

Complete f = g + h. Then list the complete pop-order for each strategy.

Node	g	h	f = g + h
A	4	3	
B	2	5	
C	7	1	
D	3	3	

Greedy ranking: _____ > _____ > _____ > _____

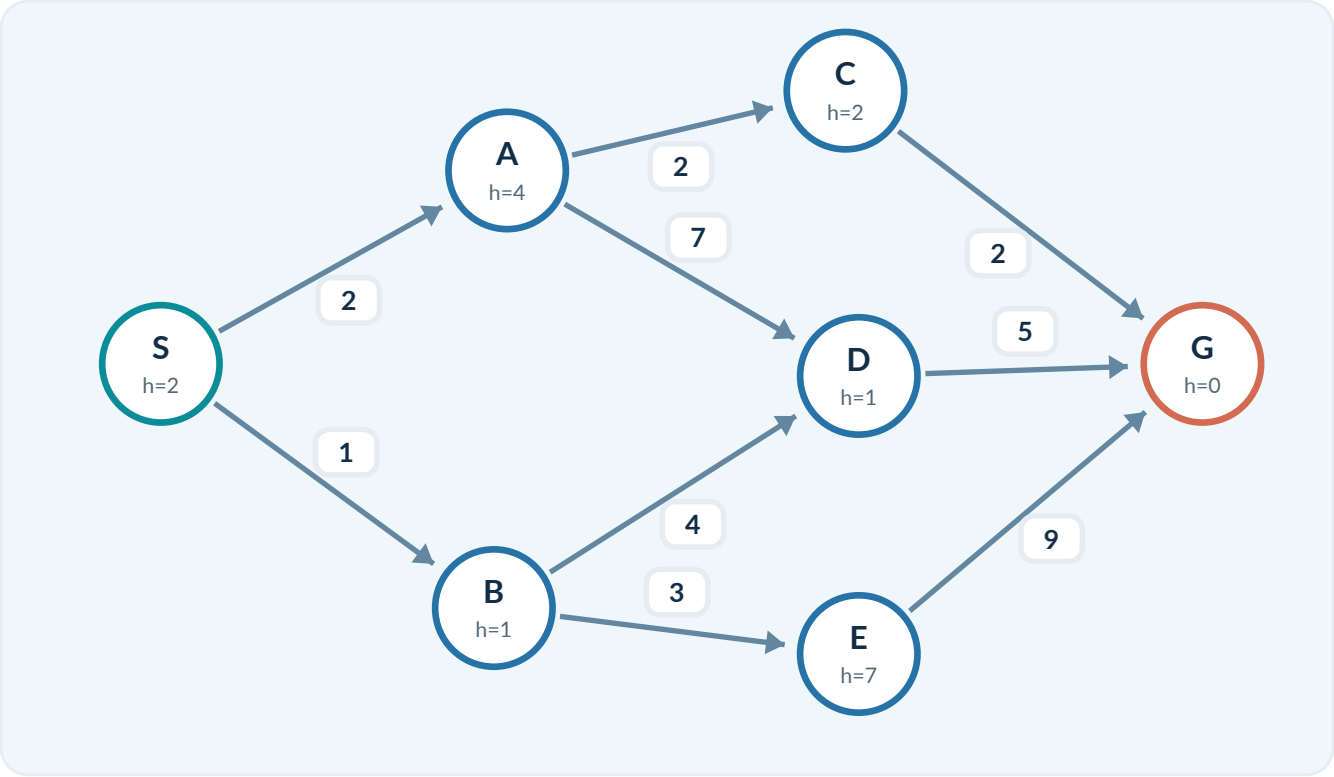
A* ranking: _____ > _____ > _____ > _____

Trace a directed weighted graph

3 Greedy Best-First on Graph A

TRACE

From S to G, trace OPEN using priority h(n) only. Edge numbers are travel costs; h values are inside each node.



Record the popped node, its accumulated g, and OPEN after adding successors.

Write OPEN in pop-order (example: B, A, D); show only each node's best record.

Step	Popped	g	h	OPEN after expansion
1				
2				
3				
4				
5				

Path S -> G: _____

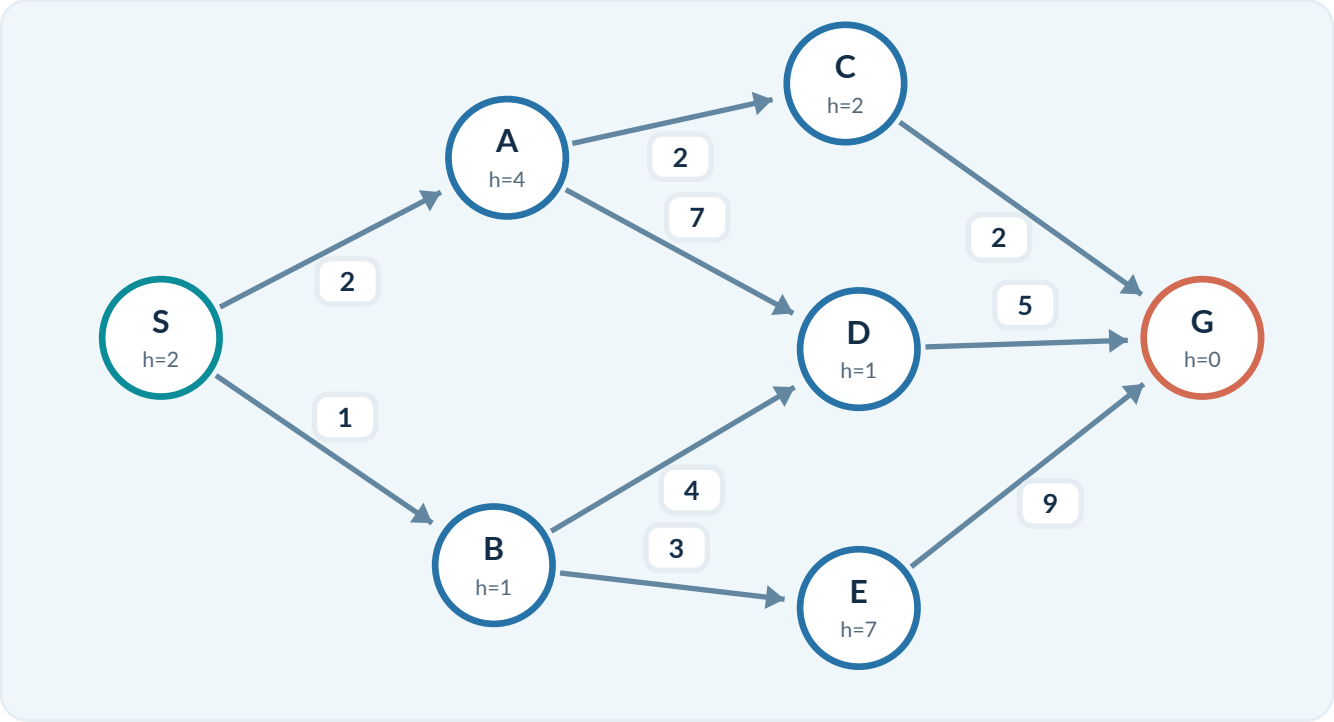
Total path cost: _____ Is this necessarily optimal? Explain. _____

Use path costs and heuristic values

4 A* on Graph A

TRACE

Use the same graph, but pop the node with smallest f = g + h. Keep improved parent pointers and best g values.



Record OPEN in increasing (f, alphabetical-name) order after each expansion.

Step	Popped	g	h	f	OPEN after expansion
1					
2					
3					
4					
5					
6					
7					

Optimal path: _____

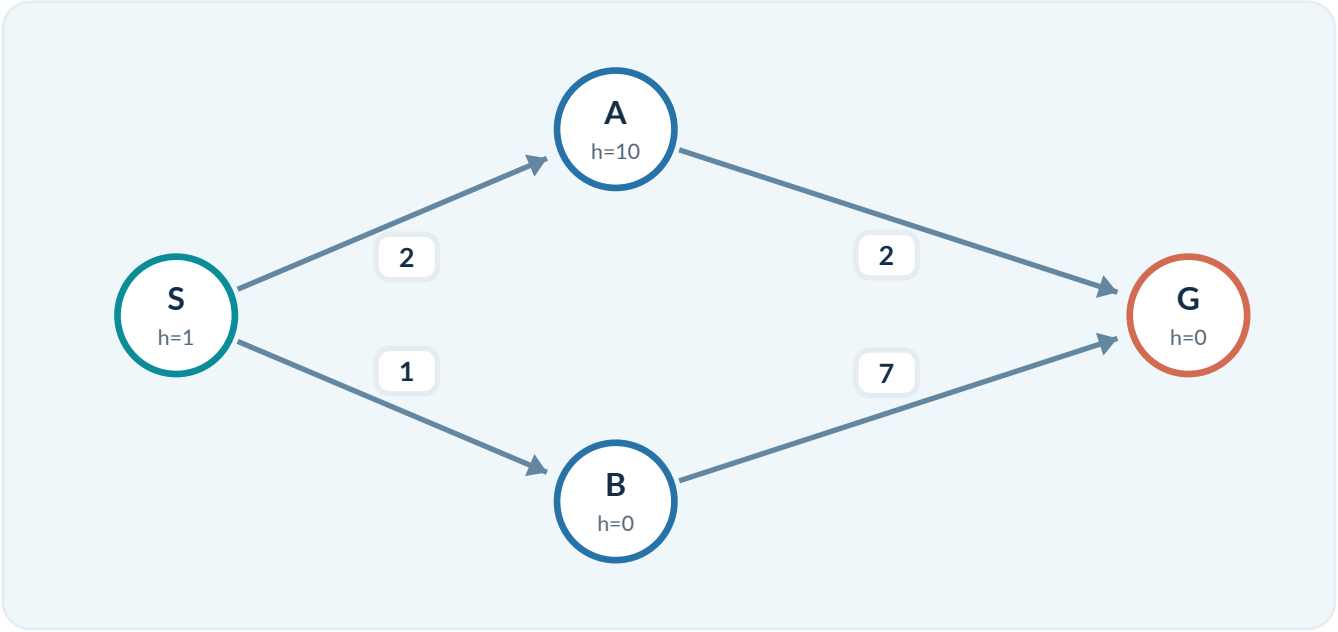
Total cost: _____ Which algorithm finds a cheaper route here? _____

When an estimate is too high

5 An overestimating heuristic

COMPARISON

Run both algorithms on Graph B. This time h(A) is intentionally unreliable. Record the pop-order, chosen path, and total cost.



Greedy Best-First

Step	Pop	g
1		
2		
3		
4		

A* Search

Step	Pop	f
1		
2		
3		
4		

Greedy path / cost:

A* path / cost:

Find the truly cheapest S -> G route and its cost:

Which h value violates admissibility? Give its true remaining cost:

Explain in one sentence why A* is not guaranteed optimal with this h:

Priority queues and improved routes

6 Sort the OPEN list

SHORT ANSWER

Entries below are already in OPEN. Sort by each algorithm's priority.

Node	g	h	f (fill in)
A	6	1	
B	2	4	
C	4	2	
D	1	4	

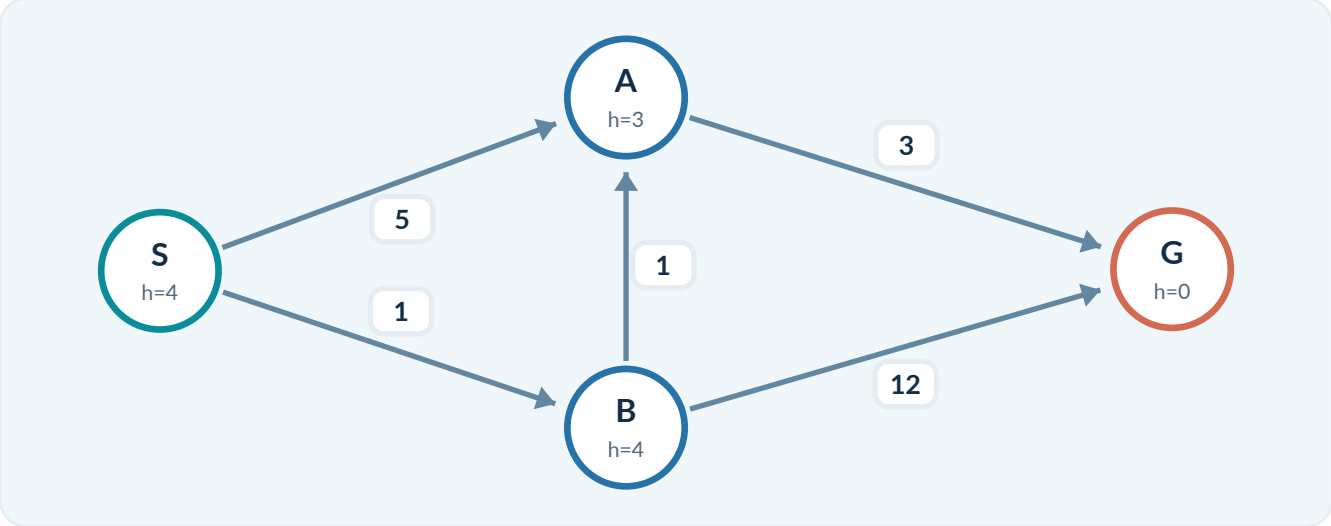
Greedy order: _____ > _____ > _____ > _____

A* order: _____ > _____ > _____ > _____

7 Update a cheaper discovered route

TRACE

Graph C: run A* from S to G. When B finds a better path to A, replace the old g(A) and parent(A).



Trace OPEN (node, f) after expansion. Do not retain stale queue records.

Step	Popped	g	OPEN after expansion
1			
2			
3			
4			

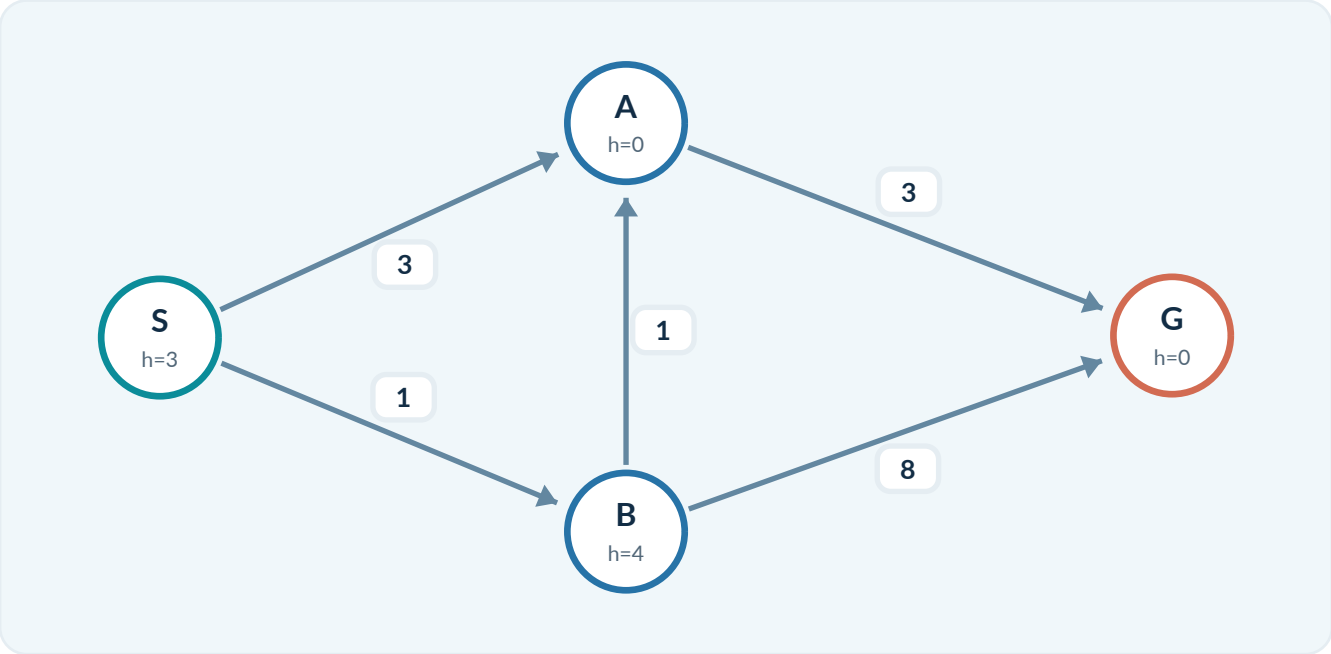
Final route: _____ Cost: _____

Inconsistent heuristics and reopening

8 A* must reopen a CLOSED node

CHALLENGE

Graph D has an admissible but inconsistent heuristic. Use the stated reopen rule when a strictly better g is discovered.



Complete the trace; the same node may appear in the Popped column twice.

Step	Popped	g	h	f	OPEN after expansion
1					
2					
3					
4					
5					
6					

Which node is reopened? _____ Old g: _____ New g: _____

Result with reopening: path _____ cost _____

If CLOSED nodes cannot reopen, which inferior result would be returned?

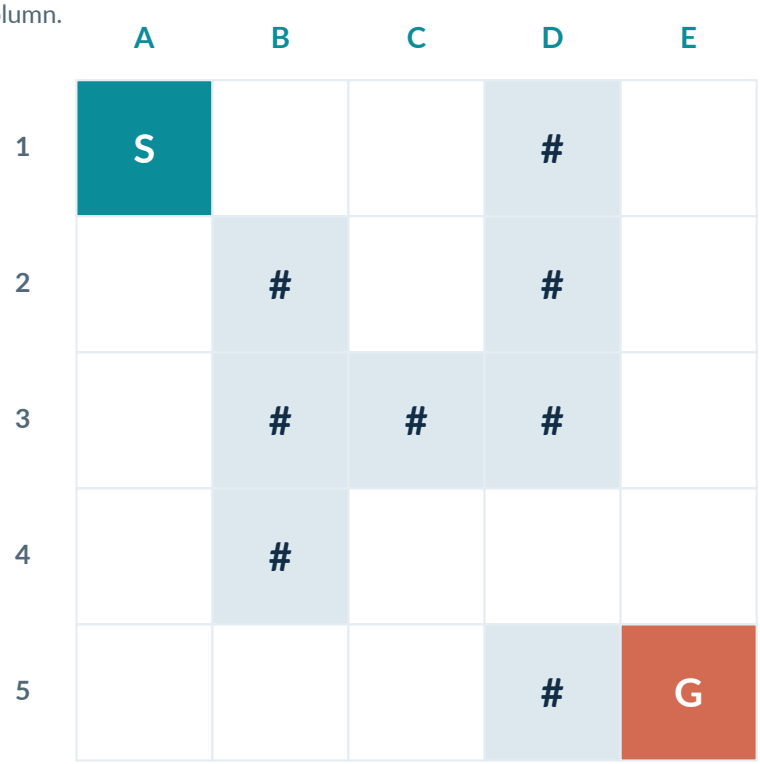
Identify an edge that violates consistency: _____

Manhattan distance and obstacles

9 Navigate a blocked grid

APPLIED

Move up/down/left/right; cost 1 per move. # is blocked. S = A1 and G = E5. Use Manhattan h and grid ties by row, then column.



HEURISTIC

h = |col - E|

+ |row - 5|

Remember: Manhattan ignores walls.

Fill estimated distances:

h(A1) = ____

h(C2) = ____

h(C5) = ____

h(E4) = ____

Mark the shortest path on the grid, then answer:

Shortest path (cell sequence):

Path cost: _____ Manhattan h(S): _____ Are they equal? _____

First six popped cells (include S):

Greedy: _____ > _____ > _____ > _____ > _____ > _____

A*: _____ > _____ > _____ > _____ > _____ > _____

Why can Manhattan underestimate the actual remaining distance?

Admissibility vs. consistency

10 Audit two heuristic functions

ANALYSIS

All edges below are directed. First calculate the true shortest distance d*(n,G); then evaluate the two heuristic columns.

DIRECTED EDGES

From	To	Cost
S	P	1
S	R	2
P	Q	2
P	G	5
R	Q	2
R	G	8
Q	G	3

ESTIMATES

Node	h1	h2
S	6	5
P	4	5
R	6	4
Q	3	1
G	0	0

Check all outgoing edges

TRUE REMAINING COSTS

d*(S)	d*(P)	d*(R)	d*(Q)	d*(G)

Classify each estimate (write Yes / No):

	h1	h2
Admissible?		
Consistent?		

Which node(s) have h1 > d*?

Give a directed edge violating consistency for h1:

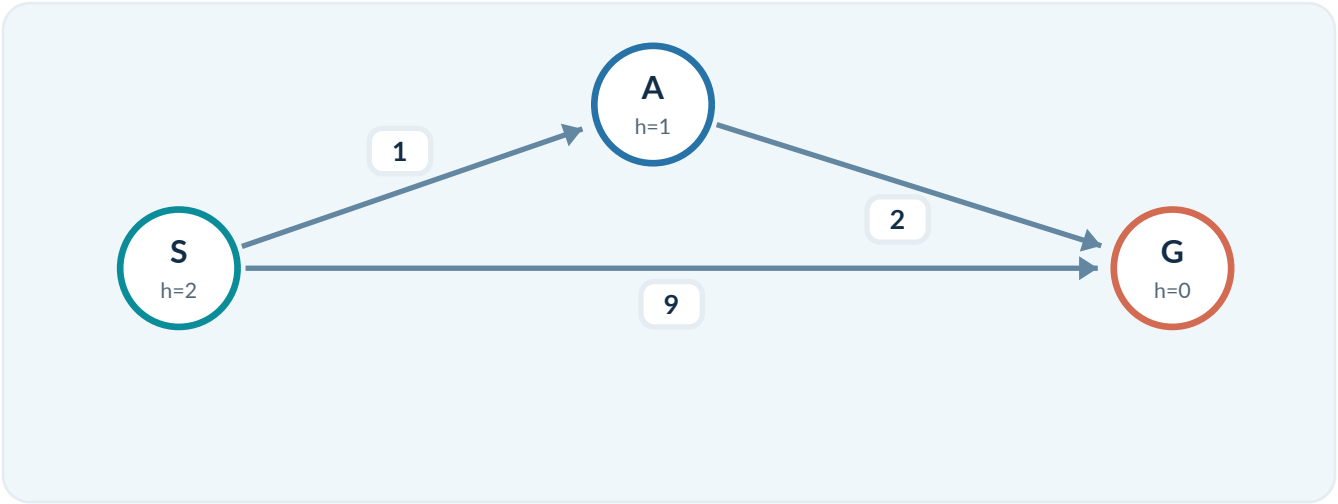
Give a directed edge violating consistency for h2:

Stopping rules and heuristic design

11 Do not stop when G is generated

DEBUGGING

An implementation stops at the first discovery of G. Show why it is wrong for this directed graph (use A*).



Early-stop path and cost: _____

Correct A* pop order: _____

Correct best path and cost: _____

12 Judge candidate grid heuristics

CONCEPT

Assume 4-direction unit-cost movement to goal G, with possible blocked cells.

Heuristic	Admissible?	Consistent?
h(n) = 0		
h(n) = Manhattan(n,G)		
h(n) = 2 x Manhattan(n,G)		
h(n) = max(0, Manhattan(n,G)-1)		
h(n) = Euclidean straight-line distance		

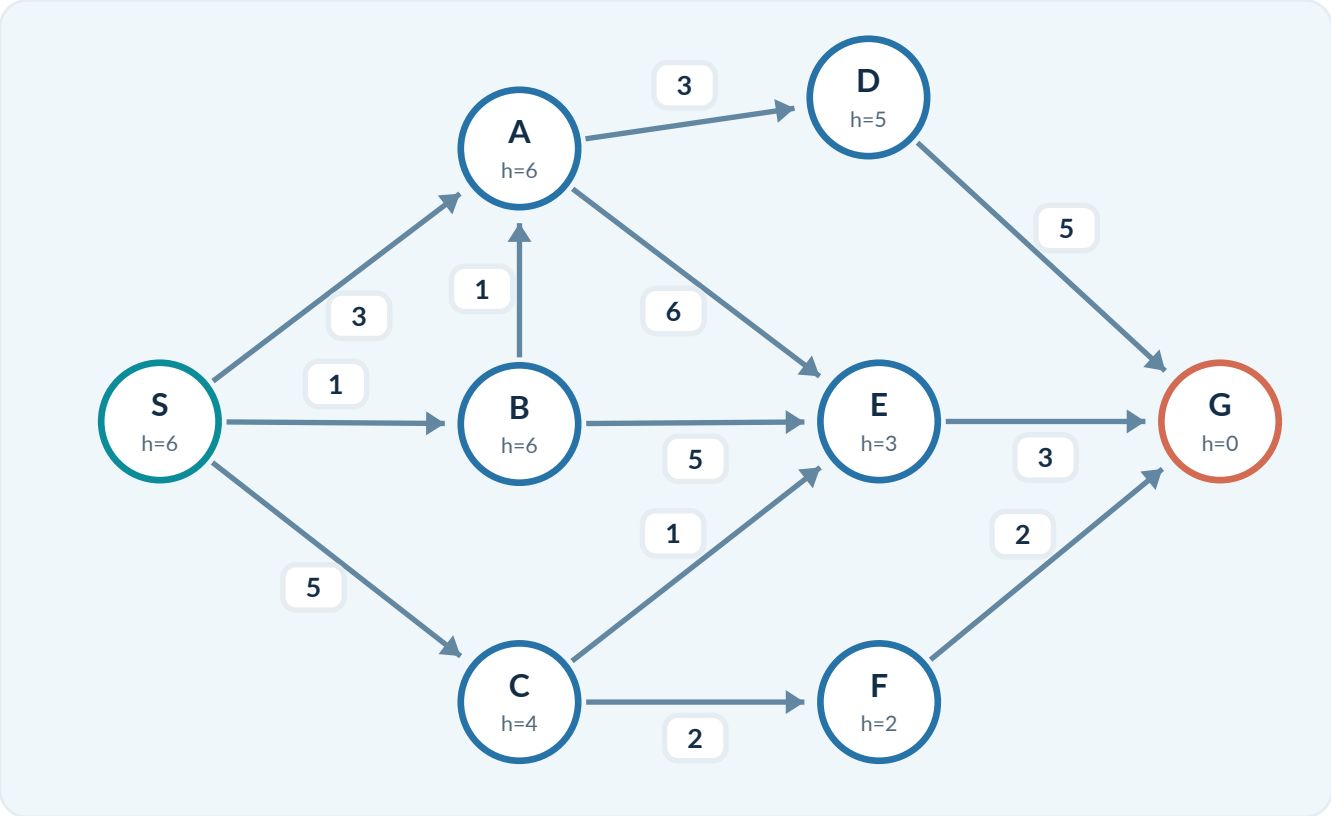
One sentence: what is the key difference between admissible and consistent?

Complete an A* run and explain ties

13 Capstone: Graph E

FINAL CHALLENGE

Trace A* from S to G. A better route to A is found later, and more than one optimal route exists.



List popped nodes and each state's g and f; use alphabetical tie-breaking.

Step	Popped	g	f	Best parent
1				
2				
3				
4				
5				
6				
7				

Expansion order: _____

Returned path and cost: _____

A different equally optimal route: _____

Question 2: Campus Medical Courier

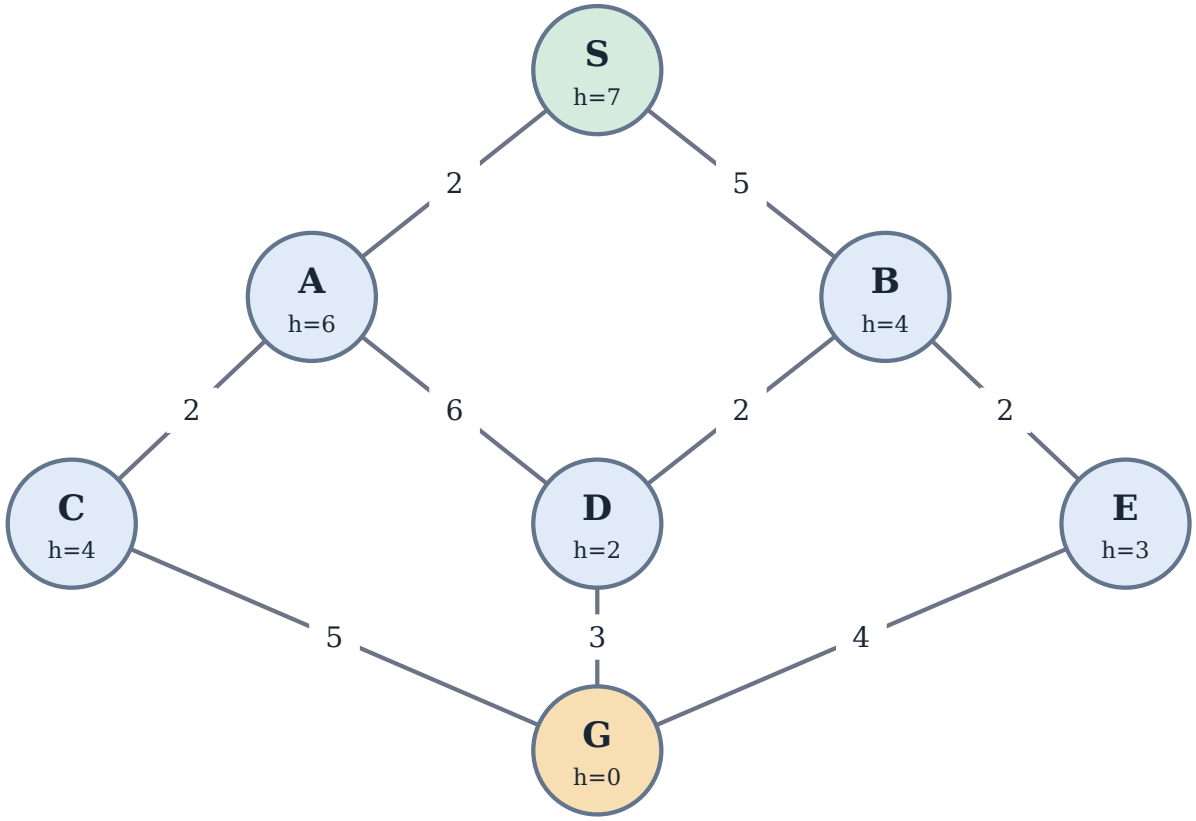
A campus courier robot must transport a sealed first-aid kit from supply point S to clinic G. Junctions are represented by nodes, and the numbers on the connecting routes indicate travel costs. The value h(n) beside each junction is an estimate of the cost still required to reach the clinic.

a) Starting at S, trace UCS until the goal is selected. List the node expansion order and identify the least-cost route to G with its accumulated cost.

b) Repeat the search using GBFS. Show the node expansion order, the selected route, and the total travel cost incurred.

c) Suppose the campus map becomes much larger and only a small amount of memory is available. Recommend a search method other than UCS or GBFS when any valid route is acceptable. Justify its memory advantage and discuss one reason it might perform poorly.

d) Using this network, explain whether the smallest heuristic value necessarily identifies the cheapest complete route. Support your explanation with the results of your two searches.



Edge labels: travel cost | h: estimated remaining cost to G

Search conventions: Treat all edges as bidirectional. Break priority ties alphabetically. Use graph search with an explored set; keep a cheaper path if a frontier node is reached again. Stop when G is removed from the frontier. Include G as the final entry in the reported expansion order.

Question 3: Underground Inspection Vehicle

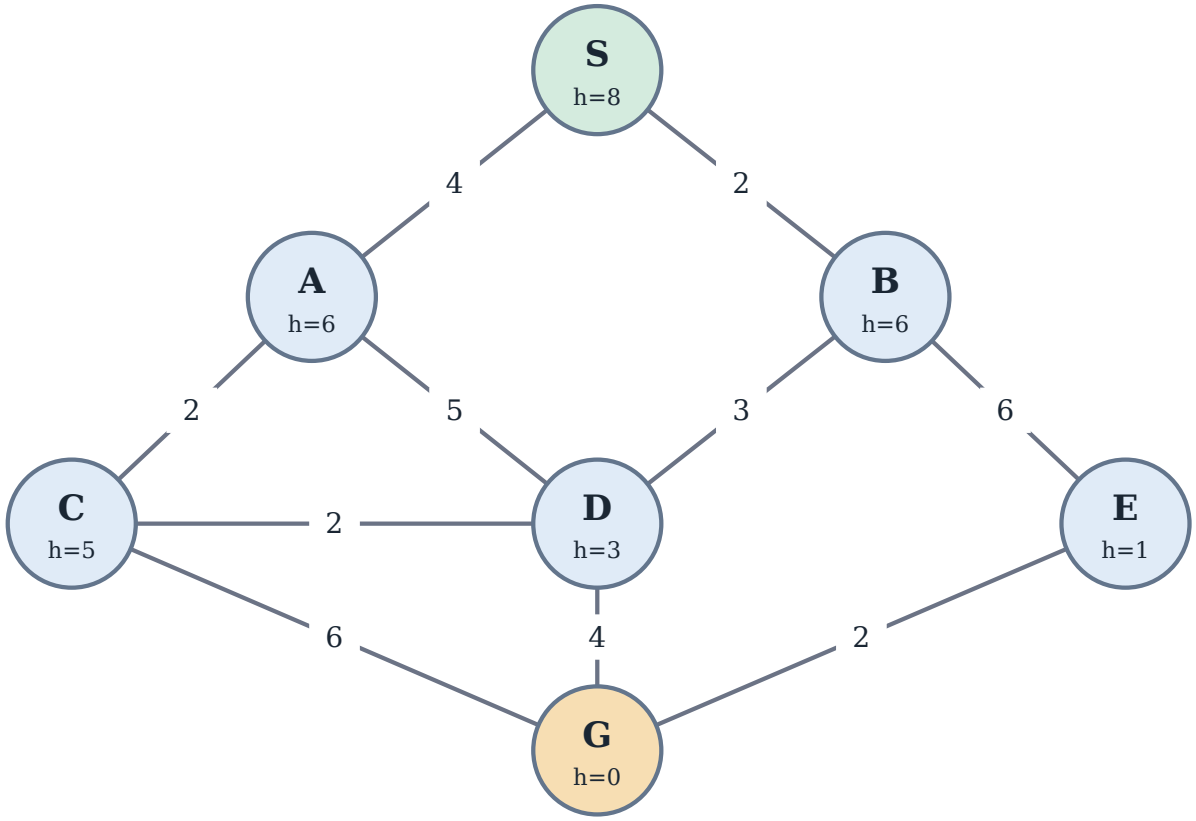
An inspection vehicle must travel from tunnel entrance S to monitoring chamber G. The graph represents accessible passages, with edge weights giving movement costs. The heuristic value at each node estimates the remaining cost to the chamber. All passages are available throughout the search.

a) Apply UCS to obtain a route from S to G. Record the node expansion order, show the final path, and determine its total movement cost.

b) Apply GBFS to the same graph using the displayed heuristic values. Record the node expansion order and the path found, including its total cost.

c) The vehicle may later explore a much deeper tunnel network with a strict memory limit. Propose an alternative strategy when optimality is not essential. Explain its memory requirements and a limitation related to deep branches or cycles.

d) Explain how UCS and GBFS prioritize a frontier node. Would altering only the heuristic values change the behavior of both methods? Give a reason for each method.



Edge labels: travel cost | h: estimated remaining cost to G

Search conventions: Treat all edges as bidirectional. Break priority ties alphabetically. Use graph search with an explored set; keep a cheaper path if a frontier node is reached again. Stop when G is removed from the frontier. Include G as the final entry in the reported expansion order.

Question 4: Harbour Maintenance Robot

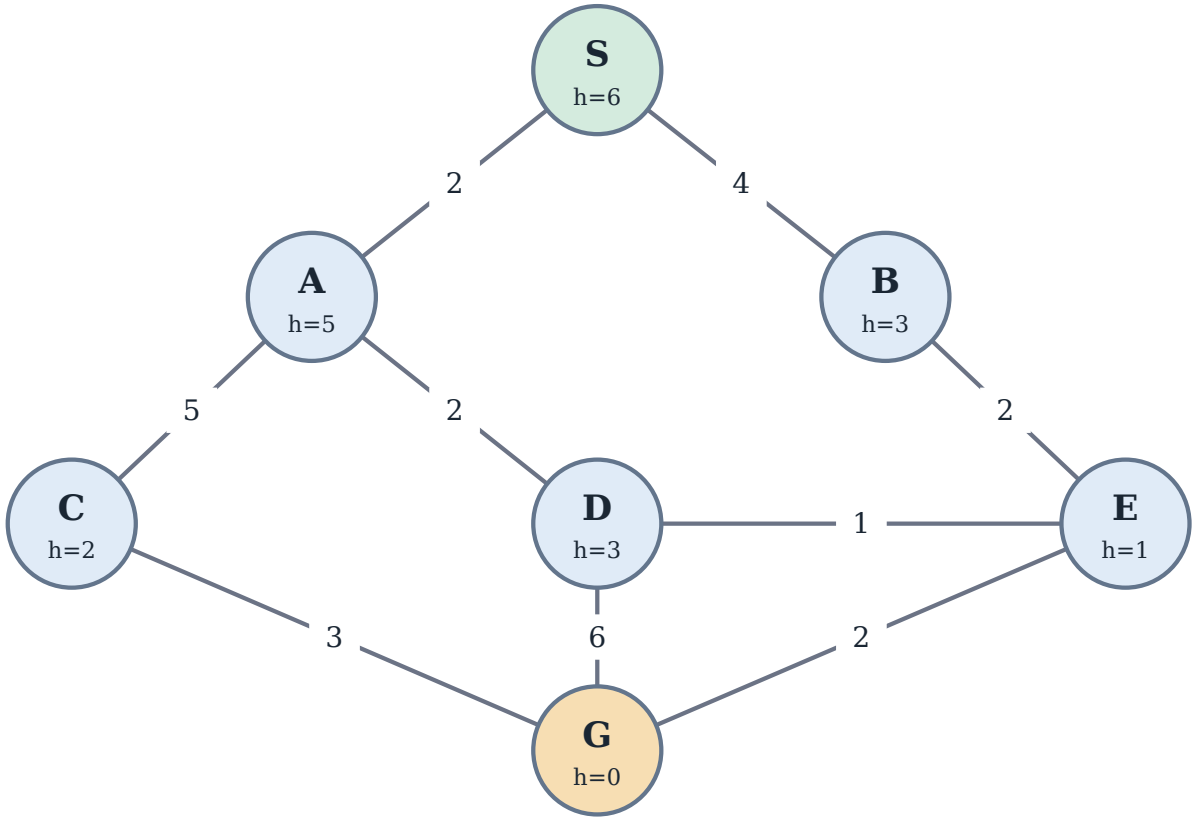
A maintenance robot moves from equipment depot S to service platform G through a harbour facility. Nodes represent access points, and edge weights represent the cost of moving between them. The listed heuristic values estimate the remaining travel cost to G; they do not represent costs already incurred.

a) Demonstrate UCS from S to G. Provide the node expansion order, the chosen route, and the final accumulated path cost.

b) Demonstrate GBFS using the given estimates. Provide the node expansion order and the resulting route, and compute its path cost.

c) For a much larger harbour network, the designer wants to reduce memory consumption and is willing to accept a non-optimal route. Recommend a different search strategy, explain why it can save memory, and state one important limitation. Does lower memory use guarantee faster discovery of a route?

d) For each algorithm, explain what should happen if a node already in the frontier is reached again by a cheaper path. Relate your explanation to the distinction between path cost and heuristic value.



Edge labels: travel cost | h: estimated remaining cost to G

Search conventions: Treat all edges as bidirectional. Break priority ties alphabetically. Use graph search with an explored set; keep a cheaper path if a frontier node is reached again. Stop when G is removed from the frontier. Include G as the final entry in the reported expansion order.